

From Jupyter Notebooks to Workflow Graphs in Bioinformatics

Static Reconstruction of Notebook Dataflow for BioFlow-Insight

Reina Al Masri

Internship report

Supervised by George Marchment and Sarah Cohen-Boulakia

June 2026

1 Introduction

Bioinformatics uses computation to analyse biological data. A complete analysis is rarely a single command: it is usually a chain of steps. This chain is often called a workflow. A workflow can include data cleaning, quality control, transformation, statistical modelling, visualisation, and interpretation. A workflow is useful not only because it executes an analysis, but because it explains how a result was obtained and helps another person rerun, inspect, reuse, or adapt the analysis.

Workflow systems such as Nextflow were created to make these analyses more reproducible by encoding dependencies in workflow code [2]. However, this structure is still not always easy to inspect directly from the implementation. BioFlow-Insight builds on this idea by statically analysing Nextflow workflow code and reconstructing several graph views of its structure without executing the workflow [1]. The role of these graphs is central: they turn dependencies that are hidden in code into an explicit structure that can be read, checked, and reused. Nodes show workflow steps, while edges show how results flow between them. These graphs can then be read by MetroFlow, which represents workflows as interactive metro maps, making process dependency graphs easier to explore [3, 4].

Many bioinformatics analyses, however, are not written as formal workflows. They are often developed in Jupyter notebooks. The Jupyter Notebook format stores an ordered document made of cells, including markdown cells, code cells, and saved outputs [5]. This format is useful for exploration and explanation, but the notebook interface does not make the underlying data dependencies explicit. A variable may be defined in one cell and used several cells later. A function may be declared once and called elsewhere. A result may depend on several previous cells. The visual order of a notebook is therefore not enough to explain its actual computational structure.

JetBrains downloaded 9.72M public Jupyter notebooks from GitHub in October 2020 [6]. The 2023 JetBrains Data Science survey also reports notebooks as a common tool for exploration, experimentation, prototyping, and presenting work; about 40

This wide use makes notebook visualisation and dependency recovery increasingly important. The motivation of this internship is therefore graph-based: if graphs help

make workflow code understandable and reusable, then similar graph views can also help reveal the hidden structure of notebooks.

This internship investigated the following guiding question:

To what extent can static analysis recover a useful workflow-like dataflow representation from Python Jupyter notebooks, while remaining compatible with BioFlow-Insight graph outputs?

The prototype follows the pipeline below:

```
.ipynb → cleaned code cells → AST parser  
cell facts → dependency graph → DOT / PNG / JSON
```

The notebook file is first reduced to cleaned code cells. These cells are parsed with the Python AST parser, then converted into cell facts such as definitions, uses, imports, calls, and conditions. These facts are used to build the dependency graph, which is exported as DOT, PNG, and JSON files.

2 Cell-Level Dependency Graph

The first graph representation is a flat cell-level graph. Each retained code cell becomes one node. A directed edge from Cell A to Cell B means that Cell B uses a variable whose most recent previous definition was produced in Cell A. This section starts with controlled examples that were used to define the modelling rules.

2.1 Examples

The controlled examples were used twice. First, they were drawn by hand to define the expected behaviour. Then they became real input notebooks used to check the implementation.

The first example is a simple assignment-chain example, containing only variable assignments and variable uses. One accepted code cell becomes one graph node, and every edge corresponds to a definition-use relation between cells, following the dependency idea used in prior notebook maintenance work [8].

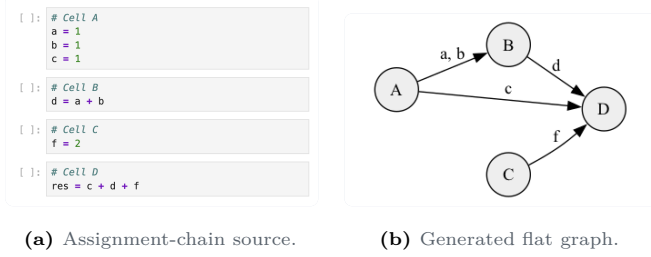


Figure 1: Assignment-chain example.

The second example introduces an import, a helper function, and a conditional call. Imports and function definitions do not create dependency edges by themselves, while values passed into calls and conditions still create dependency edges.

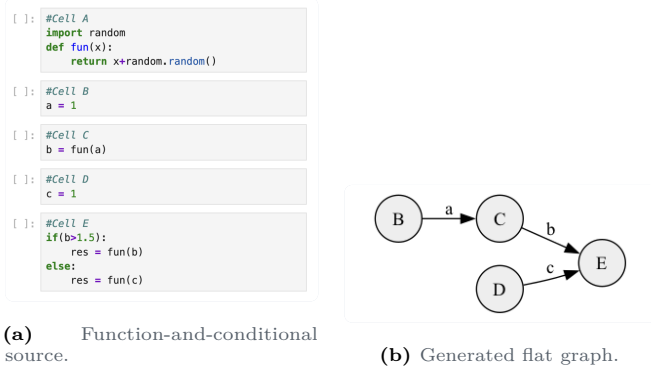


Figure 2: Function-and-conditional behaviour.

The external-function example isolates external variables read inside a function body. Cell B defines `f`, whose body reads `a`; Cell C calls `f`. The generated graph contains an edge from Cell A to Cell C labelled `a`, but no edge from Cell B to Cell C solely because `f` was defined there.

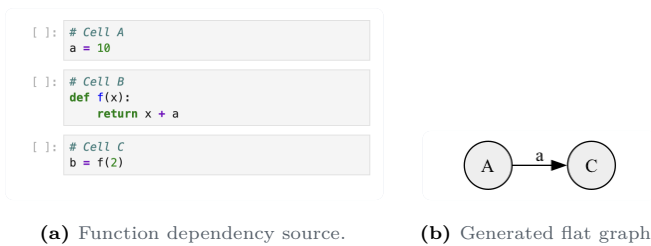


Figure 3: External function-body dependency.

The next example checks latest-definition behaviour after redefinition.

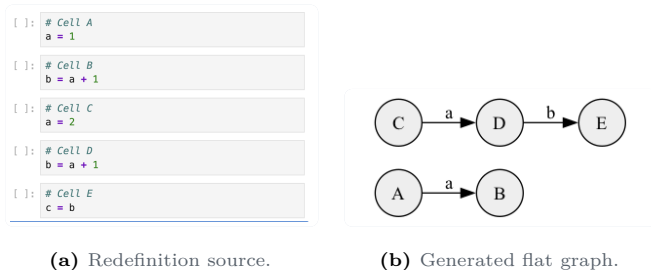


Figure 4: Redefinition example.

2.2 Cell-level modelling rules

From these examples, the flat cell-level graph follows the rules below:

- Markdown cells, empty cells, invalid cleaned Python cells, and non-informative display cells are not graph nodes.
- Whole-cell magics are discarded, while line magics and shell commands are removed before AST parsing.
- One accepted code cell becomes one graph node.
- Imports are kept as metadata, but imported names are ignored when dependency edges are built, as in previous notebook-maintenance dependency graphs [8].
- Pure import cells and pure callable-definition cells can be omitted as graph nodes.
- Function definitions are still analysed before omission, so external variables read inside function bodies remain available at later call sites.
- A data edge from Cell A to Cell B is created when Cell B uses a variable whose latest previous definition is in Cell A.
- Redefinition is handled with a latest-definition table: a later assignment replaces the previous producer for that name.
- A function name by itself does not create an edge from the definition cell to the call cell.
- If a called function body reads an external variable, the call cell depends on the latest producer of that external variable.

3 Statement-Level Dependency Graph

A cell-level graph can be too coarse. In notebooks, a single cell may contain an entire analysis phase. If one graph node represents that whole cell, the user cannot see the internal dataflow between statements. For this reason, the prototype also builds an expanded statement-level graph.

In the example below, the expanded graph contains two statement nodes inside the Cell A box, with an edge from A.1 to A.2 labelled `a`. This shows why the graph representation is useful: it does not only show that a cell exists, but also how statements inside the cell depend on each other.

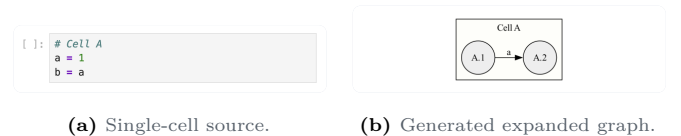


Figure 5: Expanded graph for a single cell containing two dependent statements.

This example gives the following statement-level rules:

- one accepted code cell becomes one subworkflow box;
- one accepted top-level statement becomes one graph node;
- dependencies are computed using the same latest-definition principle at statement level;

3.1 Conditional producers

Top-level `if` statements require special handling because a variable may be defined by one branch or another depending on the runtime condition. The static graph does not know which branch will execute, so it represents both possibilities.

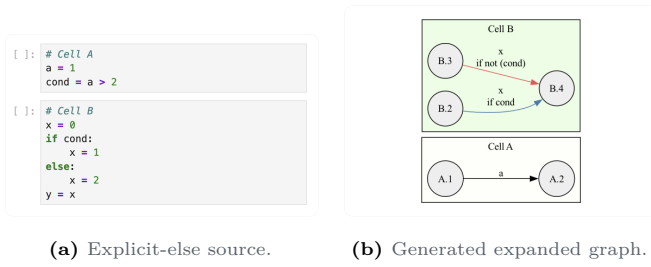


Figure 6: Conditional redefinition with an explicit `else` branch. Edge labels include both data names and branch conditions.

The statement `y = x` may depend on the producer `x = 1` under condition `cond`, or on `x = 2` under condition `not (cond)`.

A missing `else` is also meaningful:

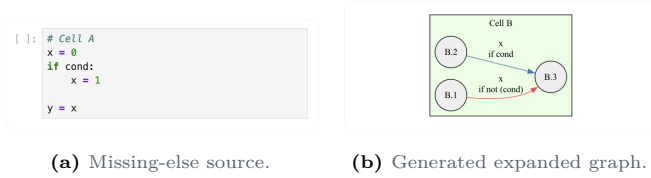


Figure 7: Conditional redefinition without an `else` branch. The false path preserves the previous producer.

Here, `y = x` can depend on `x = 1` when `cond` is true, but it can also depend on the older definition `x = 0` when `cond` is false.

Nested top-level `if` statements combine conditions. For example, a statement inside `if cond1` and then inside `if cond2` receives the condition `cond1 and cond2`. If a previous definition is kept because an inner condition is false, the graph can produce conditions such as `cond1 and not (cond2)`.

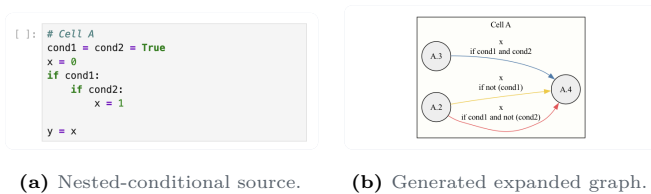


Figure 8: Nested conditional example, where conditions are merged along branch paths.

4 Validation

Validation combined controlled tests with graph generation on example notebooks. The tests checked the central rules: cell nodes, definition-use edges, DOT, JSON, and PNG generation, and stable statement identifiers. In total, the prototype was run on 32 notebooks: 19 controlled or extra example notebooks and 13 bioinformatics notebooks from the TrisKast DataScience-Bioinformatics collection. Figure 9 shows one larger example generated from the Day34 GWAS notebook from that collection [9].

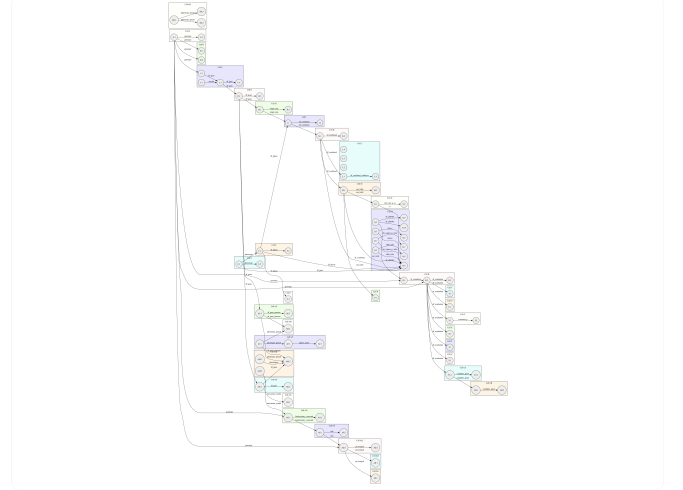


Figure 9: Expanded graph generated for the Day34 GWAS notebook, used as a larger bioinformatics example.

The second example, shown in Figure 10, is a hand-written notebook used to test conditional edges and cell subworkflows.

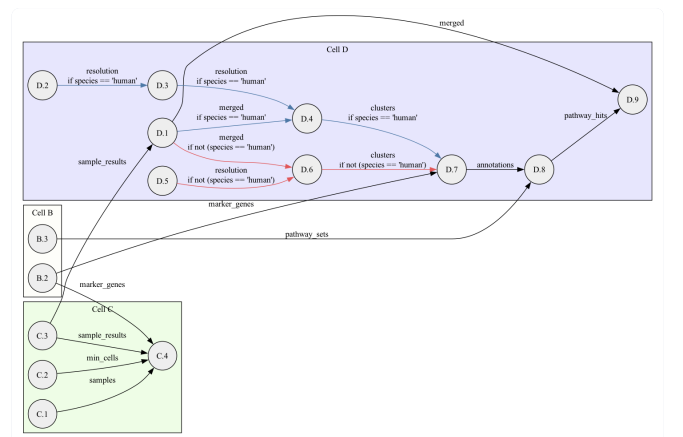


Figure 10: Expanded graph generated for the hand-written `bio2` notebook, used to test conditionals and subworkflows.

Finally, Figure 11 shows the same graph as Figure 2 exported in a MetroFlow visual style.

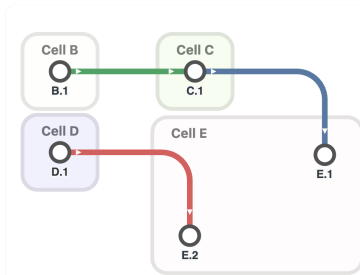


Figure 11: MetroFlow-style map produced from the notebook statement graph.

This view was obtained by exporting the reconstructed notebook graph as a JSON file compatible with the MetroFlow website [4]. The prototype computes the dependencies; MetroFlow is then used only as a visualisation layer, displaying the graph as a metro-map representation.

5 Limitations

The prototype is intentionally static. It does not execute notebooks, so it cannot know runtime values, dynamic imports, shell-created data, hidden global state, or the real order in which a Jupyter user executed cells. The current scope is one Python notebook at a time; external files, imported modules, other notebooks, and data files are not reconstructed.

Conditions are expanded only for top-level `if` structures. Loops, `try/except`, `match`, and `with` statements are analysed for names but displayed as single statement nodes.

Some dependencies also remain approximate at the Python-object level. The prototype mainly follows variable names, so object mutations such as `df["col"] = x`, in-place dataframe operations, or calls such as `model.fit(X, y)` are not tracked precisely. The current labels are still technical identifiers, and statement nodes are grouped by notebook cell rather than by biological or computational meaning.

6 Conclusion

This internship showed that static AST analysis can recover useful notebook dataflow without executing the notebook: cross-cell dependencies, latest definitions, external variables read through function calls, intra-cell dependencies, and conditional producers. The result is a notebook frontend that produces both flat and expanded graph outputs while staying close to BioFlow-Insight conventions.

More generally, the project follows the same idea as BioFlow-Insight: graph views make hidden computational structure explicit. For notebooks, this is important because the interface shows cells, but not the dataflow between them. Representing notebooks as graphs therefore makes analyses easier to inspect, explain, and reuse.

On a personal level, this project was also very meaningful to me because it was my first research internship. I learned how to read related work, turn an informal problem into modelling rules, implement a prototype, validate it, and explain its limits. Working with such a supportive and inspiring team made the experience even more valuable. It confirmed that I enjoy research, and strengthened my motivation to continue in this direction and eventually pursue a PhD.

References

- [1] George Marchment, Bryan Brancotte, Marie Schmit, Fr'ed'eric Lemoine, and Sarah Cohen-Boulakia. BioFlow-Insight: facilitating reuse of Nextflow workflows with structure reconstruction and visualization. *NAR Genomics and Bioinformatics*, 6(3):lqae092, 2024. <https://doi.org/10.1093/nargab/lqae092>.
- [2] Paolo Di Tommaso, Maria Chatzou, Evan W. Floeden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35:316–319, 2017. <https://doi.org/10.1038/nbt.3820>.
- [3] George Marchment. MetroFlow: generating interactive metro maps directly from workflow code. Nextflow Summit video, YouTube. <https://www.youtube.com/watch?v=mgR7wQIGUD0>.
- [4] MetroFlow. MetroFlow web interface. <https://metroflow.pasteur.cloud/>.
- [5] Jupyter Development Team. The Notebook file format. https://nbformat.readthedocs.io/en/stable/format_description.html.
- [6] JetBrains Datalore. We Downloaded 10,000,000 Jupyter Notebooks From GitHub: This Is What We Learned. 2020. <https://blog.jetbrains.com/datalore/2020/12/17/we-downloaded-10-000-000-jupyter-notebooks-from-github-this-is-what-we-learned/>.
- [7] JetBrains. State of Developer Ecosystem 2023: Data Science. <https://www.jetbrains.com/lp/devecosystem-2023/data-science/>.
- [8] Yuan Jiang, Christian K"astner, and Shurui Zhou. Elevating Jupyter Notebook Maintenance Tooling by Identifying and Extracting Notebook Structures. IEEE ICSME, 2022. <https://www.cs.cmu.edu/~ckaestne/pdf/icsme22.pdf>.
- [9] TrisKast. DataScience-Bioinformatics notebooks. <https://github.com/TrisKast/DataScience-Bioinformatics/tree/master/notebooks>.